Lab 9: Structures and Unions (with Pointers)

Computer Programming () |

Duration: 2 hours (in-lab) | **Topic:** User-defined types, `->` member access, passing structs by pointer

Objectives

- Define a `struct` that groups related fields under one name.
- Access members with `.` on a value and `->` on a pointer.
- Pass structs to functions *by pointer* to avoid copying and to allow modification.
- Recognize when a `union` or `enum` is the right tool.

Quick Reference

Struct definition and member access.

```
typedef struct {
    int x, y;
} Point;

Point  p  = {3, 4};
Point *pp = &p;
p.x  = 10;            /* through a value:   "." */
pp->x = 20;           /* through a pointer: "->"  -- equivalent to (*pp).x */
```

Pass by pointer (no copy, caller can be modified).

```
void move(Point *p, int dx, int dy) {
    p->x += dx;
    p->y += dy;
}
int main(void) {
    Point p = {0, 0};
    move(&p, 3, 4);   /* p is now {3, 4} */
}
```

Why use `->`? `p->x` reads as "the `x` of the struct that `p` points to". It is purely sugar for `(*p).x` – the parentheses are needed because `.` binds tighter than `*`.

Array of structs + pointer walk.

```
Student class[N];
for (Student *s = class; s < class + N; s++) printf("%s\n", s->name);
```

Union. All members share the same memory; only one is valid at a time.
Enum. Names a small set of named integer constants.

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Point Distance + `move` via pointer *(25 min)*

Define `typedef struct { double x, y; } Point;`. Write

- `double distance(Point a, Point b)` – by value, since it only reads.
- `void move(Point *p, double dx, double dy)` – by pointer, since it writes.

Read two points, print their distance, then move the first one and print again.

In-Lab Exercise 2: Student Record – read into pointer *(30 min)*

Define a `Student` struct with `id` (int), `name` (char[20]), `gpa` (double). Write `void read_student(Student *s)` that fills the struct via `s->id`, `s->name`, `s->gpa`. Read N students ($N \leq 30$) into an array and print the one with the highest GPA.

In-Lab Exercise 3: Sort by GPA via Pointer Swaps *(35 min)*

Extend the previous exercise: sort the array in *descending* order of GPA. Use selection sort and a helper `void swap(Student *a, Student *b)` that swaps two struct values through their pointers. Print the sorted list, one student per line.

In-Lab Exercise 4: Tagged Union: Shape Area *(30 min)*

Define an enum `Kind { CIRCLE, RECT }` and a struct `Shape` containing a `Kind kind` and a `union` with the parameters of each shape (`double r` for the circle, `double w, h` for the rectangle). Write `double area(const Shape *s)` that uses a `switch` on `s->kind`.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Value copy vs. pointer

What does this print? Explain why `p` did not change.

```c
#include <stdio.h>
typedef struct { int a, b; } P;
void set_by_value(P x)   { x.a = 99; }
void set_by_ptr  (P *x)  { x->a = 99; }
int main(void) {
    P p = {1, 2};
    set_by_value(p);
    printf("%d %d\n", p.a, p.b);
    set_by_ptr(&p);
    printf("%d %d\n", p.a, p.b);
    return 0;
}
```

Trace & Predict 2: -> versus .

Predict the output:

```c
#include <stdio.h>
typedef struct { int x, y; } V;
int main(void) {
    V  v  = {7, 8};
    V *pv = &v;
    pv->x = 100;
```

```
    (*pv).y = 200;
    printf("%d %d\n", v.x, v.y);
    return 0;
}
```

Find the Bug 1: Wrong member access and missing braces

List the bugs and rewrite the corrected program:

```
#include <stdio.h>
typedef struct { int w, h; } Rect;
int area(Rect *r) { return r.w * r.h; }
int main(void) {
    Rect r = 4, 5;
    printf("%d\n", area(r));
    return 0;
}
```

Write Code on Paper 1: Complex addition via pointer

Define `typedef struct { double re, im; } Complex;` and write `void add(const Complex *a, const Complex *b, Complex *out)` that stores $a + b$ in `*out`. Write a complete program that reads two complex numbers, adds them, and prints the result as `a+bi`.

Write Code on Paper 2: Find oldest student

Define a `Student` struct with `name` and `age`. Write `Student *oldest(Student *arr, int n)` returning a pointer to the oldest student in the array. Demonstrate from `main` with three students.